

11 Atividade A-011: Descrição RTL

A descrição lógica de mais baixo nível é a *register transfer level* (RTL). Neste tipo de descrição, o objetivo é descrever o circuito apenas a nível dos componentes mais básicos (portas lógicas, latches e flip-flops). Numa descrição RTL, o código pode ser lido da mesma forma que o esquemático do circuito, isto é, ele não apresenta característica algorítmica, mas de ligação de gates e flip-flops. Para exemplificar a descrição RTL, esta atividade traz como exemplo o MUX 2:1.

Um Multiplexador 2:1 (ou MUX 2:1) é um circuito combinacional que seleciona uma de duas entradas (IN0 ou IN1) para ser transmitida à saída (OUT), com base no valor de uma linha de seleção (SEL). Ele funciona como um "interruptor eletrônico" controlado, permitindo que apenas uma entrada passe para a saída de acordo com o sinal de controle. A Tabela 4 apresenta a Tabela da Verdade para esse circuito combinacional.

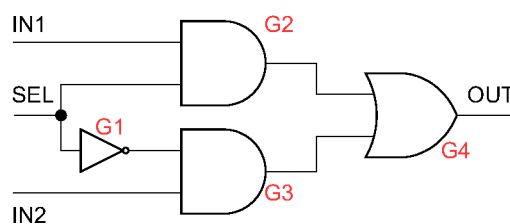
Tabela 4: Tabela da Verdade da operação do *MUX 2:1*

Entrada (IN1)	Entrada (IN2)	Seleção (SEL)	Saída (OUT)
0	X	0	0
1	X	0	1
X	0	1	0
X	1	1	1

11.1 Código Fonte (RTL)

Quando o nosso objetivo é descrever um circuito, podemos descrevê-lo no nível mais baixo através dos gates lógicos utilizados pelo circuito apresentado na Figura 8.

Figura 8: Circuito de um MUX 2x1.



O código apresentado no fragmento abaixo representa a descrição RTL do circuito multiplexador apresentado na Figura 8.

```

1 module mux2x1_rtl_1(
2     input in1, in2, select,
3     output out
4 );
5     wire n_select, and1, and2;
6
7     not g1(n_select, select);
8     and g2(and1, select, in1);
9     and g3(and2, n_select, in2);
10    or g4(out, and1, and2);
11 endmodule

```

O código seguinte descreve o mesmo circuito, porém, utilizando os operadores para representar os gates lógicos em vez de instanciar diretamente os gates.

```
1 module mux2x1_rtl_2(  
2     input in1, in2, select,  
3     output out  
4 );  
5     wire n_select, and1, and2, or1;  
6  
7     assign n_select = ~select;  
8     assign and1 = in1 & select;  
9     assign and2 = in2 & n_select;  
10    assign or1 = and1 | and2;  
11    assign out = or1;  
12  
13    // ou simplesmente:  
14    // assign out = (in1 & select) | (in2 & ~select);  
15 endmodule
```

Ambos os códigos apresentados anteriormente descrevem o mesmo circuito. O primeiro utiliza de elementos da linguagem Verilog conhecidos como primitivas, enquanto o segundo utiliza operadores lógicos.

11.2 Exercícios

Exercício 1: Implemente uma *testbench* capaz de avaliar o comportamento dos códigos fornecidos anteriormente.

Exercício 2: Faça uma pesquisa e escreva como implementar o mesmo circuito através de outro operador da linguagem Verilog.

Exercício 3: Escreva um código Verilog que represente o funcionamento do circuito combinacional da figura abaixo utilizando descrição RTL.

